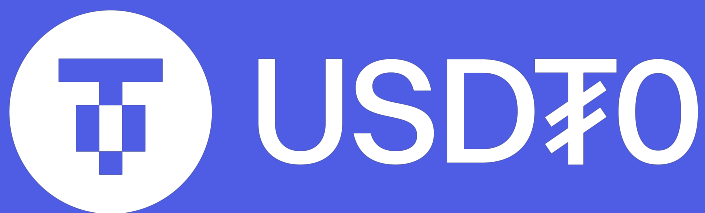


Everdawn Avalanche USDT to USDT0 Upgrade Review



August 21, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	6
The Token Upgrade	6
The Cross-Chain OFT and Backing Model	7
The Migration Process	7
Approach	8
Security Model and Trust Assumptions	9
Privileged Roles	10
Notes & Additional Information	11
N-01 Manifest Directory Is Configured After the Upgrades Plugin Reads It	11
N-02 Instantiation Path Is Reachable by Any Inbound Message and Its Event Is Unauthenticated	11
N-03 Upgrade-Preparation Script Suggests Combining the Activation Call With the Upgrade	12
N-04 Insufficient Test Coverage for the Upgrade Tooling and the OFT Changes	13
Conclusion	14
Appendix	15
Issue Classification	15

Summary

Type	Stablecoins	Total Issues	4 (4 resolved)
Timeline	From 2026-08-18 To 2026-08-21	Critical Severity Issues	0 (0 resolved)
Languages	Solidity Typescript	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	0 (0 resolved)
		Notes & Additional Information	4 (4 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed a review of the process and code changes for upgrading the native USDT token on the Avalanche C-Chain to a USDT0-compatible implementation. The token is deployed behind a [TransparentUpgradeableProxy](#) at [0x9702...A8c7](#), and the migration upgrades its implementation in place while preserving the token address, balances, allowances, and all other existing state.

The review covered the following sources:

- The [Avalanche USDT to USDT0 Upgrade Playbook](#), which describes the end-to-end migration process, provided by the client as a GitHub gist.
- The [Everdawn-Labs/usdt0-tether-contracts-hardhat](#) repository at the [a7d3145](#) commit, covering the new token implementation and its supporting upgrade tooling.
- The [Everdawn-Labs/usdt0-oft-contracts](#) repository, covering the changes to the OFT contract introduced in [pull request #225](#) at the [2c890ba](#) commit.

In scope were the following files:

```
usdt0-tether-contracts-hardhat
├── contracts/Wrappers
│   └── OFTEExtension.sol
├── contracts/Tether
│   ├── TetherTokenV2.sol
│   ├── TetherToken.sol
│   ├── EIP3009.sol
│   └── WithBlockedList.sol
├── scripts
│   └── avalanche_usdt0_prepare_upgrade.js
├── test
│   └── AvalancheUpgrade.test.js
├── docs
│   └── avalanche-usdt0-upgrade-analysis.md
└── .openzeppelin/usdt0
    └── avalanche.json

usdt0-oft-contracts
├── contracts
│   └── OUpgradeable.sol
```

The purpose of the review was to assess the correctness and safety of the in-place upgrade, the storage compatibility of the new implementation with the live proxy, the associated deployment tooling, and the migration process described in the playbook.

The following were out of scope and were not in place at the time of the review: the LayerZero receive-side security configuration (the DVN set and threshold, the receive library, and the enforced options), the off-chain accounting DVN, the Ethereum-side funding of the lockbox, and the cross-chain peer wiring. Their role in the security of the deployment is discussed under Security Model and Trust Assumptions.

System Overview

Avalanche hosts a native USDT token, issued directly by Tether and deployed behind a `TransparentUpgradeableProxy`. This engagement concerns the migration of that token into the USDT0 system. Rather than deploying a new token, the migration upgrades the existing implementation in place, so the token keeps its address and all of its current state, and connects it to a newly deployed OFT contract. Because the token is upgraded rather than redeployed, the review focused on the safety of the upgrade, the compatibility of the new implementation with the existing storage, and the process that connects the token to the cross-chain system.

This engagement was a review of the migration process and its supporting code and tooling. Where a step could be verified ahead of execution, such as the storage compatibility of the new implementation, the correctness of the upgrade transaction, and the bytecode of the already-deployed contracts, OpenZeppelin performed that verification directly, including additional testing beyond the provided suite and a full rehearsal of the migration sequence against a fork of the Avalanche network. For steps that could not be executed as part of the review, such as the Ethereum-side funding of the lockbox, OpenZeppelin confirmed that the step as described is correct and necessary, and its correct execution is relied upon as an assumption. The cross-chain security configuration and the off-chain accounting were not yet established at the time of the review, so their resulting state could not be verified directly. The role of these components in the security of the deployment is discussed under Security Model and Trust Assumptions.

The Token Upgrade

The new token implementation, `TetherTokenOFTExtension`, extends the existing Tether token with the functions required to participate in the USDT0 system. It adds an `oftContract` storage variable, a `setOFTContract` function restricted to the owner, and the `crosschainMint` and `crosschainBurn` functions defined by the ERC-7802 interface. The two cross-chain functions can only be called by the address stored in `oftContract`, and while that address is unset they revert, so the cross-chain functionality remains inactive until the owner explicitly enables it. The new implementation preserves the storage layout of the deployed token and appends its new variables after the existing ones, so balances, allowances, the blocklist, permit nonces, and the token metadata are carried over unchanged

by the upgrade. The implementation also adds the EIP-3009 transfer-with-authorization functions and retains the owner's existing ability to mint, redeem, and manage the blocklist.

The Cross-Chain OFT and Backing Model

The OFT contract, `0Upgradeable`, is the LayerZero application that sends and receives cross-chain messages. On an outgoing transfer it burns tokens from the sender through the token's `crosschainBurn` function, and on an incoming transfer it mints tokens to the recipient through `crosschainMint`. On Ethereum, USDT0 instead uses an OFT adapter, known as the lockbox, that locks and unlocks USDT, and the balance held in it backs the entire cross-chain supply. Pull request #225, modifies the incoming-transfer path of `0Upgradeable`. It redirects transfers addressed to the token contract itself to the `0xdead` address, and it adds an instantiation path in which a transfer addressed to `address(0x1)` emits an event and mints nothing. The instantiation path is intended to seed the accounting used to track the Avalanche supply when the chain joins the system.

The Migration Process

The playbook describes the migration as an ordered sequence. The token implementation is upgraded first, while its cross-chain functions remain gated. Backing for the existing Avalanche supply is then established on Ethereum by depositing an equivalent amount of USDT into the lockbox. Ownership of the token and its proxy administrator is transferred to the USDT0 multisig. The Avalanche supply is then registered with the accounting used by the system, and finally the owner connects the token to its OFT contract, which activates cross-chain transfers. The ordering is significant, as the token is funded before it is registered and registered before it is activated, so that the cross-chain supply is backed and accounted for before any transfer is possible.

Approach

OpenZeppelin reviewed the migration process, the new token implementation, the supporting deployment tooling, and the changes to the OFT contract. The following was verified directly:

- The runtime bytecode of the implementation currently deployed on Avalanche, at [0xBA2a...75A6](#), matches the source preserved in the repository as [TetherTokenAvalancheV0.sol](#).
- The storage layout of the new implementation, [TetherToken0FTExtension](#), is a strict append to the deployed layout: the existing variables keep their slots and the new variables are added after them, with no collision.
- The OpenZeppelin Upgrades manifest at [.openzeppelin/usdt0/avalanche.json](#), which the upgrade tooling uses as its validation baseline, records a proxy, administrator, implementation address, and storage layout that match the on-chain state.
- The in-scope token contracts are unchanged since the [c6bfeb8](#) commit reviewed in a previous OpenZeppelin engagement.
- The standard ERC-20 interface and its behavior are preserved, including existing allowances, permit nonces, and the EIP-712 domain separator, so existing integrations that rely on the standard interface are unaffected.

OpenZeppelin also reviewed the upgrade-preparation script and the changes introduced by pull request #225. The provided test, [AvalancheUpgrade.test.js](#), was reviewed and its state-preservation and access-control invariants confirmed sound. Observations on these are reported in the findings.

Beyond the provided test, OpenZeppelin performed additional testing of the upgraded token and rehearsed the migration end to end against a fork of Avalanche mainnet. Impersonating the token's owner, OpenZeppelin executed the upgrade, the ownership transfers, the activation, and the emergency deactivation, and confirmed that the token state was preserved and that the new functions were correctly gated. This rehearsal exercises the sequence on forked state and is not a substitute for the execution performed at cutover.

As specified in the playbook, OpenZeppelin will independently confirm at cutover that the deployed implementation's bytecode matches the reviewed commit.

The remaining migration steps, including the Ethereum-side funding of the lockbox, the cross-chain configuration and its verification, and the off-chain accounting, could not be observed during the review and are relied upon as assumptions. These steps should be verified as they

are carried out. In particular, the cross-chain configuration and wiring, comprising the peers, libraries, DVN configuration, and enforced options, are typically verified in a separate deployment or configuration review once the OFT is deployed and connected. These assumptions are described under Security Model and Trust Assumptions.

Security Model and Trust Assumptions

The cross-chain mint and burn functions on the token are controlled entirely by the address stored in `oftContract`. These functions perform no allowance or balance checks beyond confirming the caller, so the OFT contract is fully trusted: any address set as the OFT contract can mint or burn arbitrary amounts of the token. Setting this address is restricted to the owner, which makes the correct configuration of the owner and of the OFT address a central assumption of the system.

The upgrade preserves the owner's ability to mint and redeem the token directly. After the migration, the cross-chain supply is intended to be backed one-to-one by USDT held in the Ethereum lockbox, but the owner retains the ability to change the Avalanche supply outside of that backed flow. Maintaining the backing therefore depends on the owner, which becomes the USDT0 multisig after the handover, not minting outside the intended process.

The upgrade activates the EIP-3009 transfer-with-authorization functions and derives the EIP-712 domain separator from the token name. The name is assumed to remain unchanged once integrators rely on it, as changing it through `updateNameAndSymbol` would alter the domain separator and invalidate outstanding permit and EIP-3009 signatures.

The security of incoming cross-chain messages depends on the LayerZero receive-side configuration and on the accounting used by the system to track supply across chains. Both were out of scope and were not in place at the time of the review, and both are essential to protecting the deployment against forged cross-chain messages. The correctness of the token upgrade does not, by itself, secure the cross-chain deployment.

OpenZeppelin assumes that the migration steps described in the playbook are executed correctly and in the order specified. In particular, the review assumes that the lockbox is funded with an amount matching the Avalanche total supply, read at a finalized block, before that supply is registered with the system, that registration precedes the activation of cross-

chain transfers, and that the cross-chain security configuration and the off-chain accounting are established correctly. The review also assumes that no administrative transaction that changes the total supply, whether a pending mint, a redeem, or a `destroyBlockedFunds` call, is executed between the supply snapshot and the establishment of its backing. The safety of the migration depends on this ordering being followed.

Privileged Roles

At the time of the review, the token and its proxy administrator were both owned by the same account, a Gnosis MultiSig wallet at `0xD83d...d2Aa` configured to require three of six signers. This account can upgrade the token implementation through the proxy administrator and can call every owner-restricted token function, including minting, redeeming, managing the blocklist, connecting the OFT contract, and transferring ownership.

The migration transfers ownership of both the token and its proxy administrator to the USDT0 multisig at `0x4DFF...0bf8`, a Gnosis Safe configured to require three of five signers. This is the same multisig that owns the XAUT0 token on Avalanche. After the handover, this multisig holds all of the owner privileges described above.

Once connected, the OFT contract is the only address permitted to call the token's `crosschainMint` and `crosschainBurn` functions, giving it unrestricted authority to change the token supply as part of cross-chain transfers.

Notes & Additional Information

N-01 Manifest Directory Is Configured After the Upgrades Plugin Reads It

The `avalanche_usdt0_prepare_upgrade.js` script selects the OpenZeppelin Upgrades manifest directory by setting the `MANIFEST_DEFAULT_DIR` environment variable inside its `main` function. The Upgrades plugin reads this variable only once, when its module is imported, and that import is triggered by the `require('hardhat')` call at the top of the file, which runs before `main`. The assignment therefore takes effect too late, the plugin falls back to the default `.openzeppelin` directory, which does not contain the Avalanche manifest, and the run stops with a "not registered" error before it produces the upgrade transaction. Exporting the variable in the environment before running the script avoids the problem, and the layout then validates against the live proxy.

Consider setting `MANIFEST_DEFAULT_DIR` before the `require('hardhat')` call, or documenting that it must be exported in the environment, so the script runs as described in the playbook.

Update: Resolved at commit [e430230](#).

N-02 Instantiation Path Is Reachable by Any Inbound Message and Its Event Is Unauthenticated

In the OFT contract, the `_credit` function treats an inbound transfer addressed to the instantiation address (`address(0x1)`) as a special case that mints no tokens and instead emits the `LogInstantiatedOFT` event. This path places no restriction on the sender or on when it may be used, so once cross-chain transfers are active any inbound message can reach it, and the event records only a block number and an amount, without any information identifying the source chain or the sender. The path is intended to seed the accounting that tracks the Avalanche supply a single time when the chain joins the system, so an event that

can be produced repeatedly and by any sender could seed that accounting again, or with an amount that does not correspond to a genuine instantiation. The realized effect depends on how the off-chain accounting consumes the event, which was not part of this review.

Consider allowing the instantiation path to run only once, before cross-chain transfers are active, and treating later transfers to the instantiation address like other transfers that cannot be minted. Recording the source chain and the sender in the event would also let the off-chain accounting tell a genuine instantiation apart from other messages. If the current behavior is intended, consider documenting that intent in the code so that reviewers and integrators understand the path remains open by design.

Update: Resolved. The team stated:

We will not use this feature, as it adds to much complexity and does not work as intended. The corresponding PR has been closed.

N-03 Upgrade-Preparation Script Suggests Combining the Activation Call With the Upgrade

The [avalanche_usdt0_prepare_upgrade.js script](#) prints the calldata for the upgrade and, optionally, the calldata for [setOFTContract](#), the call that connects the token to its OFT contract and activates cross-chain transfers. A comment states that this second call can be combined with the upgrade when the token owner and the proxy administrator owner are the same account, which is the situation at the time of the migration. Combining the two would activate cross-chain transfers immediately after the upgrade, before the Avalanche supply has been funded in the lockbox and registered with the accounting, which the migration requires to happen first.

Consider removing the suggestion to combine the activation call with the upgrade, so that the funding and registration steps are not skipped.

Update: Resolved at commit [a886d4e](#).

N-04 Insufficient Test Coverage for the Upgrade Tooling and the OFT Changes

The tests provided for the migration leave two areas uncovered:

- Neither the [AvalancheUpgrade.test.js fork test](#) nor the [avalanche_usdt0_prepare_upgrade.js script](#) asserts that the implementation currently deployed on the proxy matches the expected baseline before the upgrade is validated and applied. If the deployed implementation were changed between the review and the execution, the validation would run against a stale baseline without detecting it.
- The changes introduced by pull request #225 to the OFT contract's [_credit function](#), namely the instantiation path and the redirect of transfers addressed to the token contract, have no accompanying tests.

Consider asserting that the deployed implementation matches the expected baseline before the upgrade is validated, and adding tests that exercise the instantiation path and the redirect introduced in pull request #225.

Update: Resolved at commit [6064ed1](#). The team stated:

For the second point, we will not use this feature, as it adds to much complexity and does not work as intended. The corresponding PR was closed.

Conclusion

OpenZeppelin reviewed the process and code changes for upgrading the native USDT token on the Avalanche C-Chain to a USDT0-compatible implementation. The review covered the new token implementation, the upgrade tooling, the changes to the OFT contract introduced in pull request #225, and the migration process described in the playbook.

The upgrade was found to be storage-compatible with the deployed token and to preserve its address, balances, and other state, and the token contracts were confirmed to be unchanged from the versions covered by previous OpenZeppelin engagements. The full sequence was rehearsed against a fork of the Avalanche network, and no critical, high, medium, or low severity issues were identified. The review raised a number of informational notes, covering the upgrade tooling, the new instantiation path in the OFT contract, and test coverage.

The cross-chain security configuration and wiring, the off-chain accounting, and the Ethereum-side funding of the lockbox were not in place at the time of the review and are relied upon as assumptions, as described in the Security Model and Trust Assumptions section. These are expected to be verified as the migration is carried out, with the cross-chain configuration and wiring typically covered by a separate deployment or configuration review.

The Everdawn team is appreciated for their support throughout the review process.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.